WEEK 4 • DAY 4

Linear Regression: House-Price Predictor

Learn • Build • Measure Error • Debug

Date	29-Oct-2026	**Duration**	5 hours
Difficulty	Intermediate	**Deliverable**	Regression notebook + metrics

1. Learning Goals

- Explain linear regression and when it is suitable.
- Identify input features (X) and the numeric target (y).
- Use train_test_split, fit(), and predict() in scikit-learn.
- Understand residuals and MAE, MSE, RMSE, and R^2.
- Compare predictions with actual values and interpret errors in rupees.
- Identify common modelling mistakes and improve the solution.

2. Concepts in Simple Words

Linear regression: A supervised-learning method that learns a relationship between input features and a numeric target, such as estimating house price from area, bedrooms, and age.

Simple vs multiple regression: Simple regression uses one input feature; multiple regression uses two or more input features.

fit(): model.fit(X_train, y_train) learns model parameters from training examples.

predict(): model.predict(X_test) uses the learned relationship to estimate values for new rows.

Residual: Actual value − predicted value. Positive means the prediction was too low; negative means it was too high.

MAE: Mean Absolute Error averages absolute errors. It is in the target's units; lower is better.

MSE: Mean Squared Error averages squared errors. Large errors have a stronger effect; units are squared.

RMSE: Square root of MSE. It is in the target's units and penalises large errors more than MAE.

R^2: Compares the model with a baseline that predicts the training target mean. On test data it can be negative; it is not a percentage accuracy score.

Caution: The small fictional dataset teaches workflow, not real-estate market behaviour. A good score here does not guarantee real-world performance.

3. Five-Hour Learning Plan

Time	Activity

Hour 1	Review regression, features, target, fit, predict, and residuals.
Hour 2	Create and inspect the house-price dataset; identify X and y.
Hour 3	Split data and train LinearRegression.
Hour 4	Calculate MAE, MSE, RMSE, and R^2; inspect predictions.
Hour 5	Debug mistakes, try one improvement, and explain results.

4. Sample Dataset: House Prices

Create house_prices.csv from the fictional rows below, or build the same data in a pandas DataFrame. All prices are in Indian rupees.

area_sqft	bedrooms	age_years	distance_km	price_inr
600	1	15	8	3000000
750	2	10	7	3800000
900	2	8	6	4600000
1000	2	5	5	5200000
1100	3	7	5	5700000
1250	3	4	4	6600000
1400	3	6	4	7200000
1500	3	3	3	8000000
1650	3	2	3	8800000
1800	4	5	2	9500000
1950	4	3	2	10400000
2100	4	2	1	11300000
2250	4	1	2	11900000
2400	5	2	1	12800000
2600	5	1	1	13900000
2800	5	0	2	14800000
800	2	18	9	3900000
1300	3	12	7	6200000
1750	3	10	6	8500000
2000	4	8	5	9900000

Features (X): area_sqft, bedrooms, age_years, distance_km.

Target (y): price_inr. Do not include price_inr in X.

5. Build Your Regression Notebook

Create one notebook named Week4_Day4_Regression_YourName.ipynb. Run all cells from top to bottom and keep outputs visible.

Step 1 — Inspect: Load the CSV using pandas. Display head(), shape, info(), describe(), missing-value counts, and duplicate count.

Step 2 — Define X and y: Use the four feature columns for X and price_inr for y. Print their shapes.

Step 3 — Split: Use train_test_split(X, y, test_size=0.25, random_state=42). Explain why the test set is held out.

Step 4 — Fit: Create LinearRegression() and call fit(X_train, y_train). Explain what fit learns.

Step 5 — Predict: Call predict(X_test). Show actual price, predicted price, and residual for every test row.

Step 6 — Metrics: Calculate MAE, MSE, RMSE, and R^2 on test predictions. Explain each metric; report MAE and RMSE in rupees.

Step 7 — Interpret: Which test row has the largest absolute error? Are predictions mostly too high or low? What does R^2 suggest, and what are its limitations?

Step 8 — Experiment: Compare with a baseline that always predicts the training-set mean, or try another split seed as a learning experiment. Do not repeatedly tune against the test set and then claim an unbiased final score.

code:

4. Sample Dataset: House Prices

Create house_prices.csv from the fictional rows below, or build the same data in a pandas DataFrame. All prices are in Indian rupees.

# area_sqft	bedrooms	age_years	distance_km	price_inr
# 600	1	15	8	3000000
# 750	2	10	7	3800000
# 900	2	8	6	4600000
# 1000	2	5	5	5200000
# 1100	3	7	5	5700000
# 1250	3	4	4	6600000
# 1400	3	6	4	7200000
# 1500	3	3	3	8000000
# 1650	3	2	3	8800000
# 1800	4	5	2	9500000
# 1950	4	3	2	10400000
# 2100	4	2	1	11300000
# 2250	4	1	2	11900000
# 2400	5	2	1	12800000
# 2600	5	1	1	13900000
# 2800	5	0	2	14800000
# 800	2	18	9	3900000
# 1300	3	12	7	6200000
# 1750	3	10	6	8500000
# 2000	4	8	5	9900000

Features (X): area_sqft, bedrooms, age_years, distance_km.

```
# Target (y): price_inr. Do not include price_inr in X.

import pandas as pd

df = pd.DataFrame({

  "area_sqft": [

    600, 750, 900, 1000, 1100, 1250, 1400, 1500, 1650,

    1800, 1950, 2100, 2250, 2400, 2600, 2800, 800, 1300, 1750, 2000

  ],

  "bedrooms": [

    1, 2, 2, 2, 3, 3, 3, 3, 3, 4,

    4, 4, 4, 5, 5, 5, 2, 3, 3, 4

  ],

  "age_years": [

    15, 10, 8, 5, 7, 4, 6, 3, 2, 5,

    3, 2, 1, 2, 1, 0, 18, 12, 10, 8

  ],

  "distance_km": [

    8, 7, 6, 5, 5, 4, 4, 3, 3, 2,

    2, 1, 2, 1, 1, 2, 9, 7, 6, 5

  ],

  "price_inr": [

    3000000, 3800000, 4600000, 5200000, 5700000,

    6600000, 7200000, 8000000, 8800000, 9500000,

    10400000, 11300000, 11900000, 12800000, 13900000,

    14800000, 3900000, 6200000, 8500000, 9900000

  ]

})

df.to_csv("house_prices.csv", index=False)

# 5. Build Your Regression Notebook

# Create one notebook named Week4_Day4_Regression_YourName.ipynb. Run all cells from top to bottom and keep outputs visible.
```

```
# Step 1 — Inspect: Load the CSV using pandas. Display head(), shape, info(), describe(), missing-value counts, and duplicate count.

print(df.head())

print("Shape:", df.shape)

X = df[["area_sqft", "bedrooms", "age_years", "distance_km"]]

y = df["price_inr"]

print("Features shape:", X.shape)

print("Target shape:", y.shape)

df.info()

print(df.describe())

print(df.isnull().sum())

print(df.duplicated().sum())

# Step 2 — Define X and y: Use the four feature columns for X and price_inr for y. Print their shapes.

X = df[["area_sqft", "bedrooms", "age_years", "distance_km"]]

y = df["price_inr"]

print("X shape:", X.shape)

print("y shape:", y.shape)

# Step 3 — Split: Use train_test_split(X, y, test_size=0.25, random_state=42). Explain why the test set is held out.

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(

   X, y,

   test_size=0.25,

   random_state=42

)

print("Training features:", X_train.shape)

print("Testing features:", X_test.shape)

print("Training target:", y_train.shape)

print("Testing target:", y_test.shape)

# Step 4 — Fit: Create LinearRegression() and call fit(X_train, y_train). Explain what fit learns.

from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()

model.fit(X_train, y_train)

print("Model training completed!")

print("Intercept:", model.intercept_)

coefficients = pd.DataFrame({

   "Feature": X.columns,

   "Coefficient": model.coef_

})

display(coefficients)

# Step 5 — Predict: Call predict(X_test). Show actual price, predicted price, and residual for every test row.

import numpy as np

y_pred = model.predict(X_test)

results = X_test.copy()

results["actual_price"] = y_test

results["predicted_price"] = y_pred

results["residual"] = y_test - y_pred

results["absolute_error"] = np.abs(results["residual"])

display(results.round(2))

# Step 6 — Metrics: Calculate MAE, MSE, RMSE, and R² on test predictions. Explain each metric; report MAE and RMSE in rupees.

from sklearn.metrics import (

   mean_absolute_error,

   mean_squared_error,

   r2_score

)

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print(f"MAE:  ₹{mae:,.2f}")
```

```
print(f"MSE:  {mse:,.2f}")

print(f"RMSE: ₹{rmse:,.2f}")

print(f"R²:  {r2:.4f}")

# Step 7 — Interpret: Which test row has the largest absolute error? Are predictions mostly too high or low? What does R² suggest, and what are its limitations?

# Test row with the largest absolute error

largest_error_row = results["absolute_error"].idxmax()

print("Test row with the largest absolute error:")

display(results.loc[[largest_error_row]])

# Check prediction direction

underpredictions = (results["residual"] > 0).sum()

overpredictions = (results["residual"] < 0).sum()

exact_predictions = (results["residual"] == 0).sum()

print("Underpredictions:", underpredictions)

print("Overpredictions:", overpredictions)

print("Exact predictions:", exact_predictions)

if underpredictions > overpredictions:

   print("Predictions are more often too low.")

elif overpredictions > underpredictions:

   print("Predictions are more often too high.")

else:

   print("Underpredictions and overpredictions are equally frequent.")

# Step 8 — Experiment: Compare with a baseline that always predicts the training-set mean, or try another split seed as a learning experiment. Do not repeatedly tune against the test set and then claim an unbiased final score

.output:area_sqft  bedrooms  age_years  distance_km  price_inr

0       600        1        15           8    3000000

1       750        2        10           7    3800000

2       900        2         8           6    4600000

3      1000        2         5           5    5200000

4      1100        3         7           5    5700000

Shape: (20, 5)
```

Features shape: (20, 4)

Target shape: (20,)

<class 'pandas.DataFrame'>

RangeIndex: 20 entries, 0 to 19

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	area_sqft	20 non-null	int64
1	bedrooms	20 non-null	int64
2	age_years	20 non-null	int64
3	distance_km	20 non-null	int64
4	price_inr	20 non-null	int64

dtypes: int64(5)

memory usage: 932.0 bytes

	area_sqft	bedrooms	age_years	distance_km	price_inr
count	20.000000	20.000000	20.00000	20.000000	2.000000e+01
mean	1595.000000	3.250000	6.10000	4.150000	8.300000e+06
std	641.728751	1.118034	4.91935	2.455391	3.485911e+06
min	600.000000	1.000000	0.00000	1.000000	3.000000e+06
25%	1075.000000	2.750000	2.00000	2.000000	5.575000e+06
50%	1575.000000	3.000000	5.00000	4.000000	8.250000e+06
75%	2025.000000	4.000000	8.50000	6.000000	1.062500e+07
max	2800.000000	5.000000	18.00000	9.000000	1.480000e+07

area_sqft 0

bedrooms 0

age_years 0

distance_km 0

price_inr 0

dtype: int64

0

X shape: (20, 4)

y shape: (20,)

Training features: (15, 4)

Testing features: (5, 4)

Training target: (15,)

Testing target: (5,)

Model training completed!

Intercept: 1308822.4688777728

6. Code Starter (Complete the TODOs)

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("house_prices.csv")

features = ["area_sqft", "bedrooms", "age_years", "distance_km"]
X = df[features]
y = df["price_inr"]

# TODO 1: Split the data
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.25, random_state=42
)

# TODO 2: Train the model
model = LinearRegression()
model.fit(X_train, y_train)

# TODO 3: Predict test prices
y_pred = model.predict(X_test)

# TODO 4: Calculate metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R²:", r2)

# TODO 5: Show actual, predicted, and residual values
results = pd.DataFrame({
"actual": y_test,
"predicted": y_pred,
"residual": y_test - y_pred
})
display(results)
```

7. Debugging: Identify and Correct Mistakes

A. Wrong target: A student includes price_inr in X and also uses it as y.

Your task: Explain why this leaks the answer into the inputs and correct X

Answer: This leaks the answer into the inputs because price_inr is the value the model is supposed to predict. Remove it from X.

B. Training on test data: The student calls model.fit(X_test, y_test) and then reports a test score.

Your task: Explain the correct roles of training and test data.

Answer: The model should be fitted only on training data. The test data must be kept separate to check how well the model predicts unseen data.

C. MAE misinterpreted: The student says an MAE of ₹250,000 means the model is 250,000% accurate.

Your task: Explain what MAE actually measures.

Answer: MAE of ₹250,000 means the average absolute difference between actual and predicted prices is ₹250,000. It is an error measurement, not an accuracy percentage.

D. RMSE vs MSE: The student reports RMSE by printing MSE without taking the square root.

Your task: Correct the calculation and explain the units.

Answer: RMSE is the square root of MSE, calculated using rmse = np.sqrt(mse). MSE is measured in rupees squared (₹^{2}), while RMSE is measured in rupees (₹).

E. R^2 overclaim: The student says R^2 = 0.80 means every prediction is 80% accurate.

Your task: Explain why this interpretation is incorrect.

Answer: R^2 = 0.80 means the model explains 80% of the variation in the evaluated target values relative to the mean baseline. It does not mean every prediction is 80% accurate.

8. Explain in Your Own Words

1. What does linear regression learn from training data?
   Ans: Linear regression learns the relationship between input features and the output by finding the best-fit straight line. It uses this line to predict numerical values.

2. What is the difference between fit() and predict()?
   Ans: fit() – Learns patterns from the training data.
   predict() – Uses the learned patterns to predict outputs for new data.

3. Explain MAE and RMSE using rupees as the unit.
   Ans: MAE (Mean Absolute Error): The average absolute difference between actual and predicted values, measured in rupees.
   RMSE (Root Mean Squared Error): The square root of the average squared prediction errors, also measured in rupees.

4. Why does MSE penalise large errors more strongly than MAE?
   Ans: MSE squares each error, so large errors become much bigger. MAE uses the absolute value of each error.

5. What does R^2 tell you, and what does it not tell you?
   Ans: R^2 tells us how well the model explains the variation in the target values compared with predicting their mean. It does not tell us whether predictions are accurate in rupees or whether the model will perform well on new data.

6. Why must you evaluate on rows not used to fit the model?

Ans: We must evaluate the model on unseen test data to check how well it predicts new values. Evaluating only on training data may give an overly optimistic result.

9. Daily Deliverable & Checklist

Submit ONE notebook: Week4_Day4_Regression_YourName.ipynb

- Dataset loading and inspection outputs are visible.
- X and y are defined correctly.
- Train/test split is completed before fitting.
- LinearRegression is fitted on training data only.
- Predictions are generated for test rows.
- MAE, MSE, RMSE, and R^2 are calculated and explained.
- Actual-vs-predicted table includes residuals.
- Debugging answers and own-words explanations are included.
- Notebook runs from top to bottom without unexplained errors.

10. Assessment Rubric (100 Marks)

Assessment area	**Marks**
Dataset inspection and correct X/y definition	10
Correct split and model fitting	20
Predictions and residual analysis	15
Correct MAE, MSE, RMSE, and R^2 calculations	25
Debugging and interpretation	15
Own-words explanations and notebook clarity	15
TOTAL	**100**

Trainer note: This fictional dataset is small. Assess workflow and metric interpretation rather than treating its test score as reliable evidence for real housing markets.